Operaciones de Aprendizaje Automático III

Rejection Sampling Fine-tuning

MSc. Fernando Silva



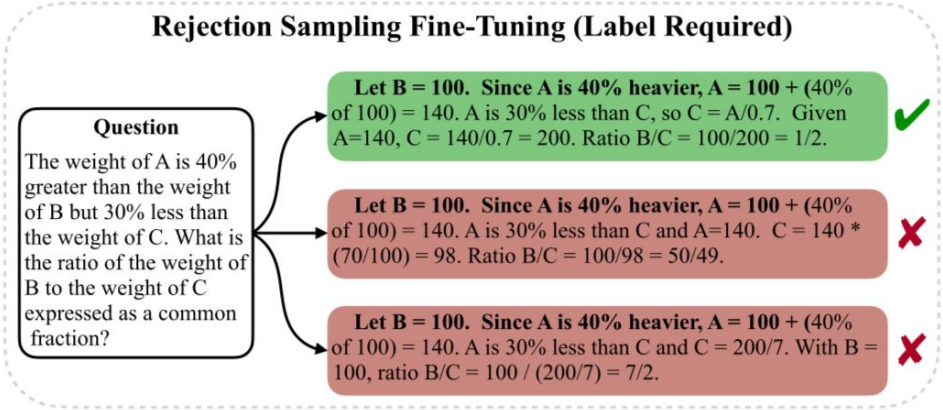
Agenda

- Rejection Sampling Fine-tuning
- text-to-sql

Rejection Sampling Fine-tuning

Rejection Sampling Fine-tuning

- RFT es un método para mejorar un modelo de lenguaje entrenándolo con sus propias respuestas correctas.
  - Rejection Sampling (muestreo con rechazo): generar muchas respuestas y descartar las incorrectas.
  - Fine-Tuning (ajuste fino): reentrenar al modelo con las respuestas que quedaron.

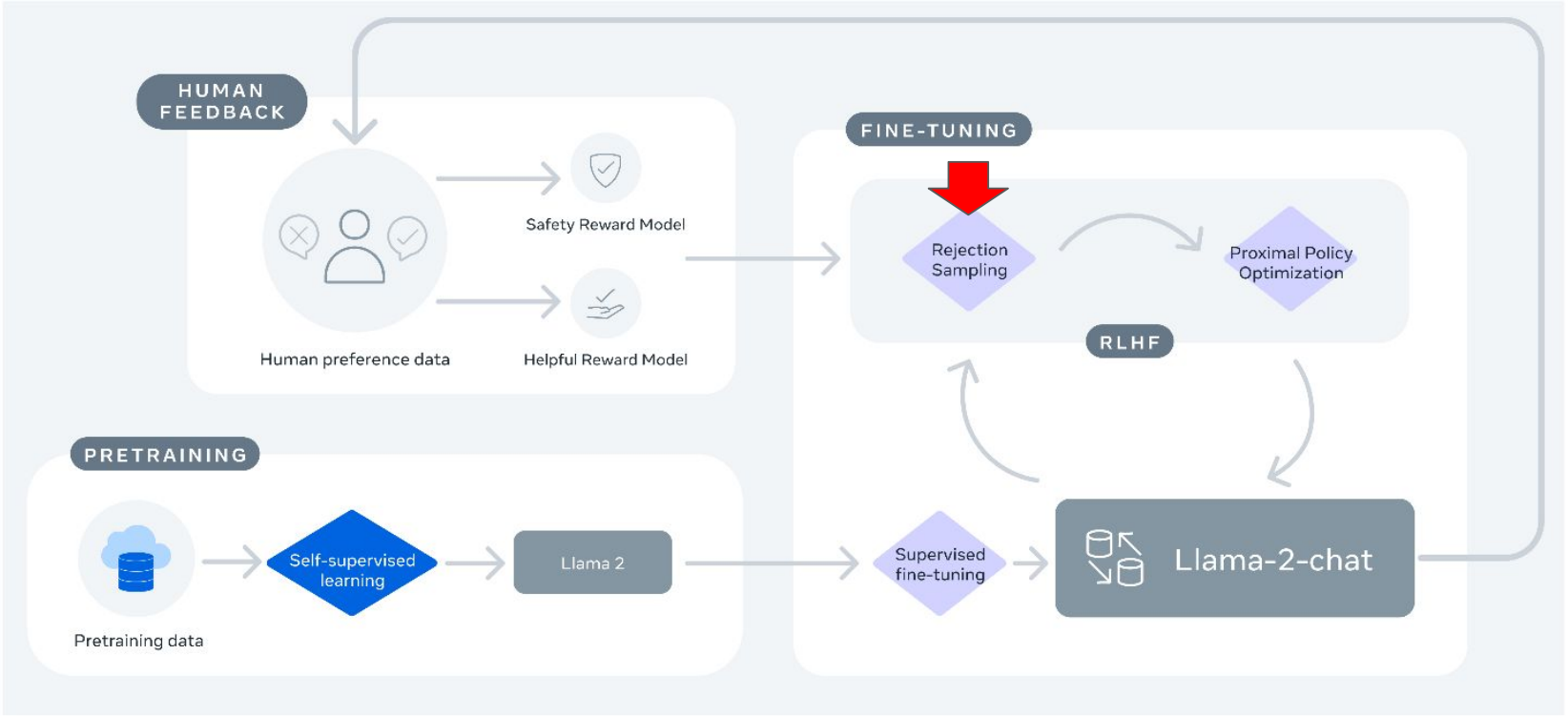


Rejection Sampling Fine-tuning

- **1. Muestrear:** para cada pregunta, el modelo genera varias respuestas usando azar, así cada una sale distinta.
- **2. Verificar:** cada respuesta se compara con la solución correcta, que ya se conoce (juez)
- **3. Rechazar:** las incorrectas se descartan. Las correctas se guardan en un dataset de pares (pregunta, respuesta correcta).
- **4. Entrenar:** se hace un fine-tuning supervisado normal con ese dataset, para que el modelo aumente la probabilidad de producir esas respuestas.

Rejection Sampling Fine-tuning

En Llama 2, RFT sirvió para que el modelo aprendiera a dar sus mejores respuestas: generaba varias, un modelo de recompensa elegía la mejor y se entrenaba con ella, ronda tras ronda





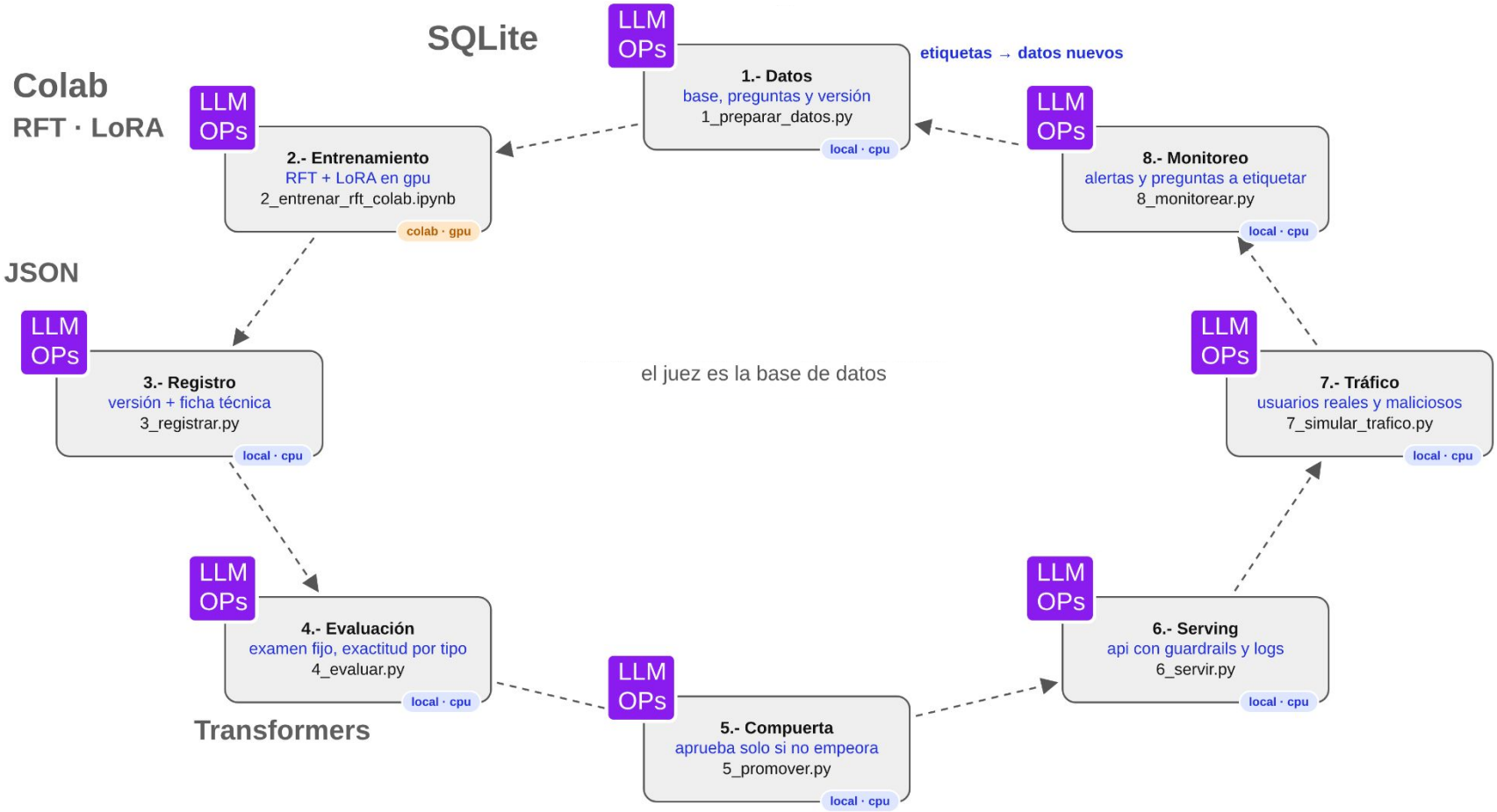
Ejercicio

RFT

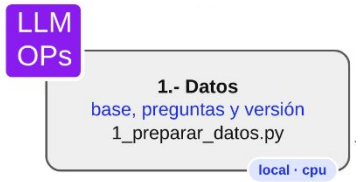
LINK

text-to-sql

Flujo


SQLite
Colab
RFT · LoRA
JSON
Transformers
LLM OPs
1.- Datos
base, preguntas y versión
1_preparar_datos.py
local · cpu
etiquetas → datos nuevos
2.- Entrenamiento
RFT + LoRA en gpu
2_entrenar_rft_colab.ipynb
colab · gpu
3.- Registro
versión + ficha técnica
3_registrar.py
4.- Evaluación
examen fijo, exactitud por tipo
4_evaluar.py
5.- Compuerta
aprueba solo si no empeora
5_promover.py
6.- Serving
api con guardrails y logs
6_servir.py
7.- Tráfico
usuarios reales y maliciosos
7_simular_trafico.py
8.- Monitoreo
alertas y preguntas a etiquetar
8_monitorear.py
el juez es la base de datos

Flujo


LLM
OPs
1.- Datos
base, preguntas y versión
1_preparar_datos.py
local · cpu

Flujo

A: “¿Cuántos estudiantes aprobaron Estadística?”, que está en el examen.

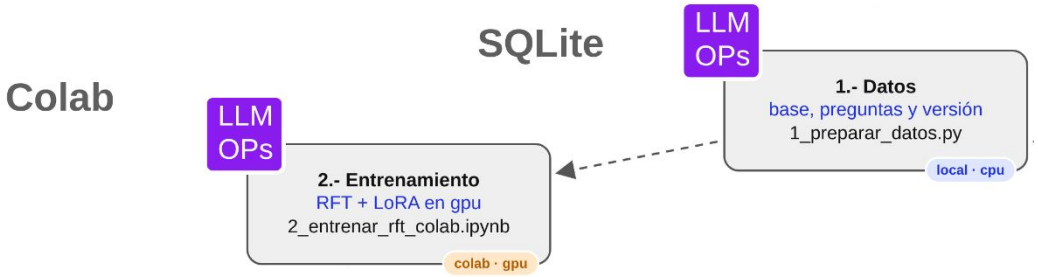
B: “ciudades ordenadas por cantidad de estudiantes”, que llega desde producción.

Se crea la base (estudiantes, cursos, notas) y se generan las preguntas con su SQL correcta

- A queda guardada en el examen (prueba.json):

{
"tipo": "agregar",
"pregunta": "¿Cuántos estudiantes aprobaron Estadística?",
"sql": "SELECT COUNT(*) FROM notas n JOIN ... WHERE c.nombre = 'Estadística' AND n.nota >= 51"
}

Flujo


Colab
SQLite
LLM OPs
1.- Datos
base, preguntas y versión
1_preparar_datos.py
local · cpu
LLM OPs
2.- Entrenamiento
RFT + LoRA en gpu
2_entrenar_rft_colab.ipynb
colab · gpu

Flujo

A: “¿Cuántos estudiantes aprobaron Estadística?”, que está en el examen.

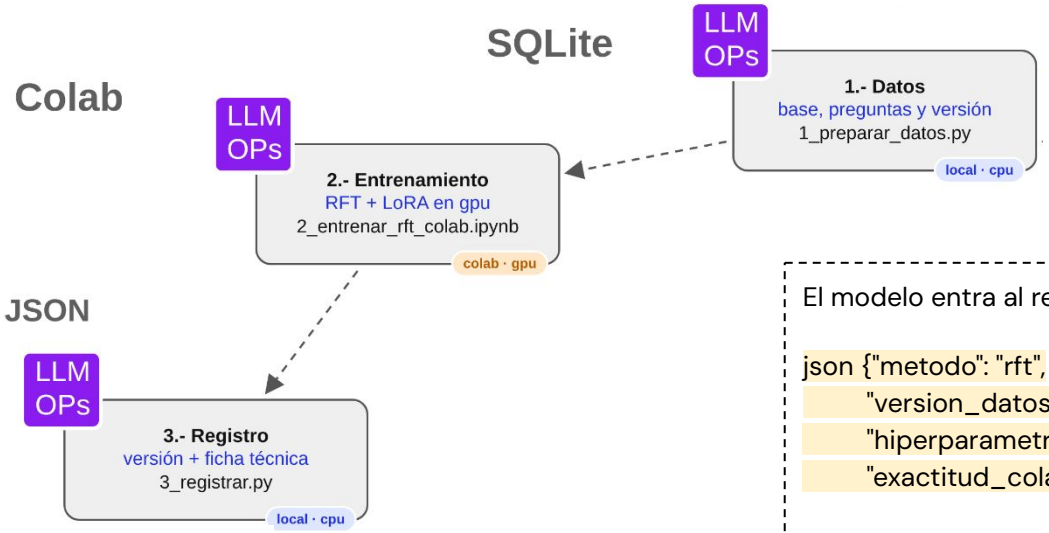
B: “ciudades ordenadas por cantidad de estudiantes”, que llega desde producción.

Para una pregunta parecida de entrenamiento “¿Cuántos estudiantes aprobaron Física?” el modelo intenta 8 veces, por ejemplo:

```
NO: SELECT COUNT(*) FROM notas WHERE curso_id = 2  -> 20  (olvidó la nota)
OK: SELECT COUNT(*) FROM notas n JOIN cursos c ... AND n.nota >= 51 -> 13
```

- el juez ejecuta cada intento y compara el resultado
- los OK se quedan y se entrenan con LoRA
- tras 2 rondas, la exactitud pasó de 28 % a 61 % a 94 %
- de acá sale modelo_rft.zip

Flujo

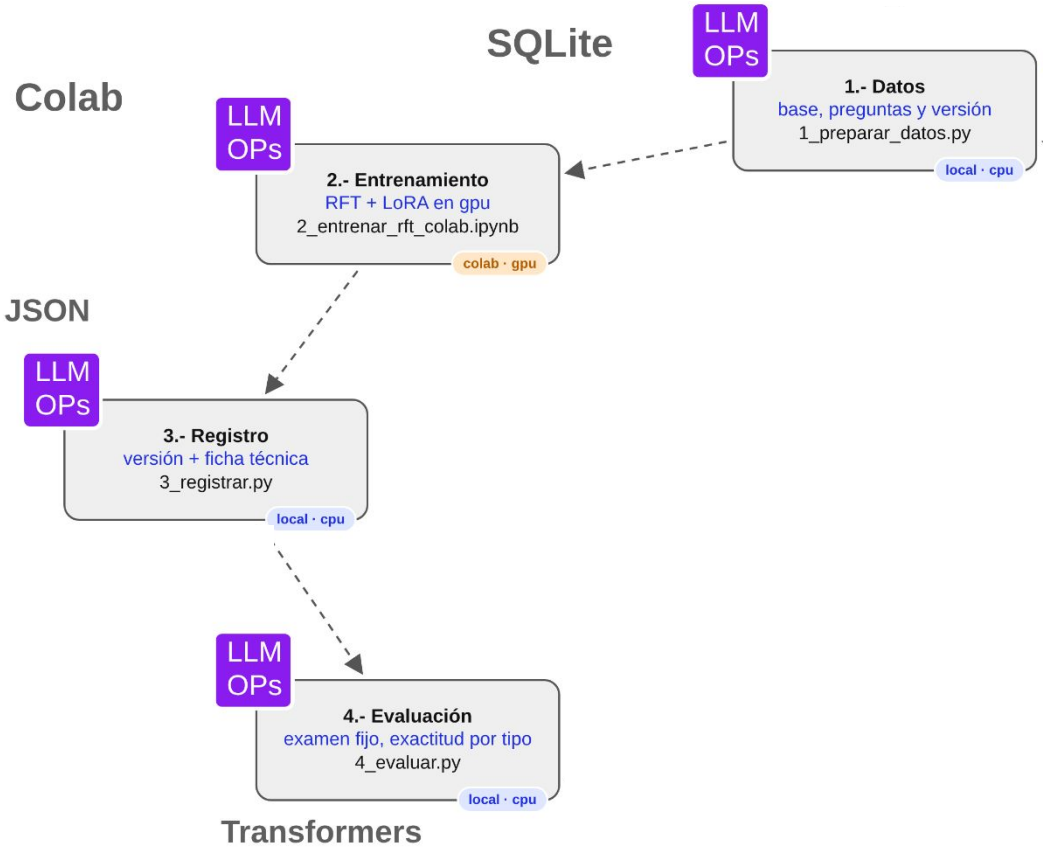


El modelo entra al registro como v1, con su ficha:

```
json {"metodo": "rft",
      "version_datos": "c4d9e2b6",
      "hiperparametros": {"rondas": 2, "respuestas": 8, ...},
      "exactitud_colab": [0.28, 0.61, 0.94]}
```

asi se sabe siempre con que datos y con qué configuración se entrenó

Flujo


SQLite
LLM OPs
1.- Datos
base, preguntas y versión
1_preparar_datos.py
local · cpu
Colab
LLM OPs
2.- Entrenamiento
RFT + LoRA en gpu
2_entrenar_rft_colab.ipynb
colab · gpu
JSON
LLM OPs
3.- Registro
versión + ficha técnica
3_registrar.py
local · cpu
LLM OPs
4.- Evaluación
examen fijo, exactitud por tipo
4_evaluar.py
local · cpu
Transformers

Flujo

A: “¿Cuántos estudiantes aprobaron Estadística?”, que está en el examen.

B: “ciudades ordenadas por cantidad de estudiantes”, que llega desde producción.

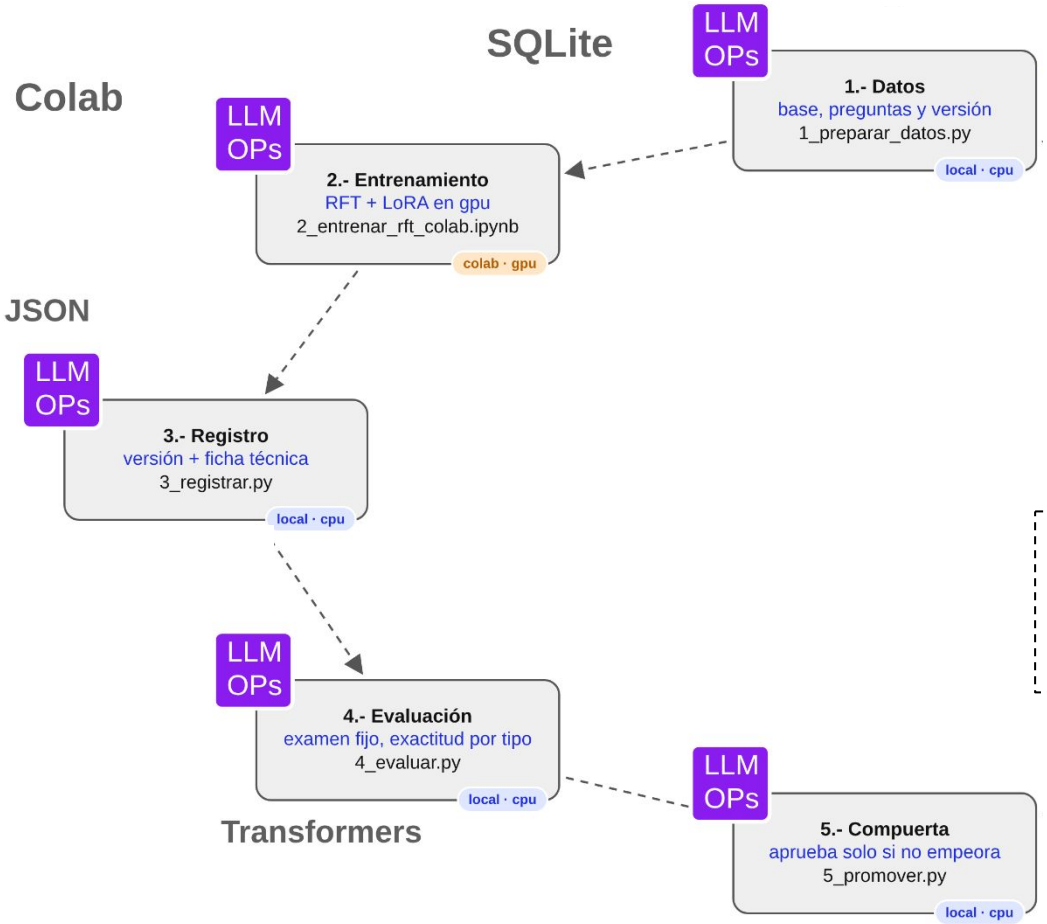
- la evaluación son 18 preguntas fijas (prueba.json) que el modelo nunca vio al entrenar (el A)

- 4_evaluar.py carga el modelo de manera local, le hace las 18 preguntas y el juez marca cada respuesta OK o NO, la exactitud es aciertos / 18

- tenemos:
  - v0 falla “aprobaron Estadística” y acierta 5 de 18 (28 %)
  - v1 la acierta y llega a 17 de 18 (94 %)

- quien entrena no debe ser quien aprueba, el examen local revisa el modelo que realmente llegó en el zip, en la máquina donde va a correr, y ese es el número que usa la compuerta

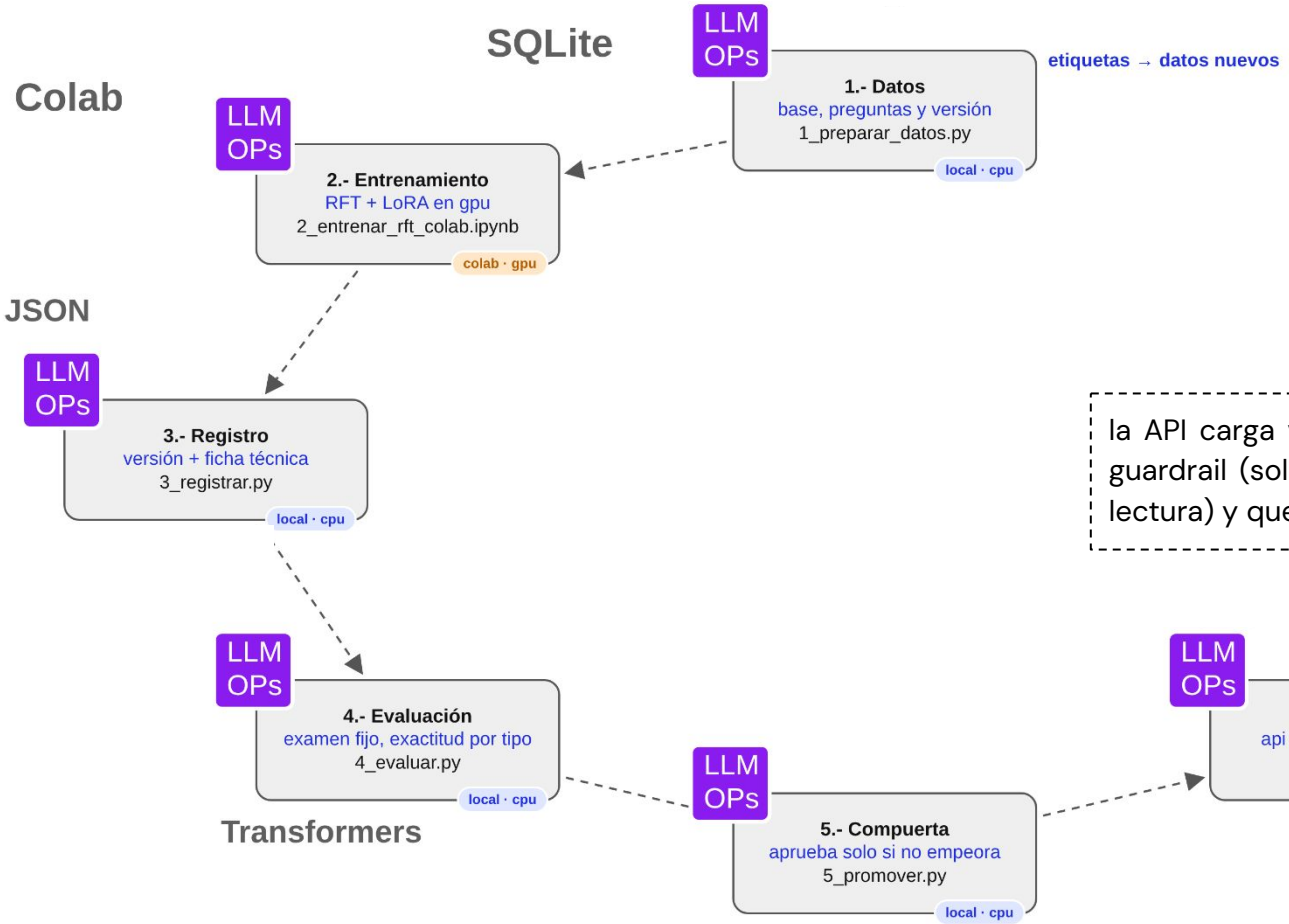
Flujo



v1 se compara con producción (v0):

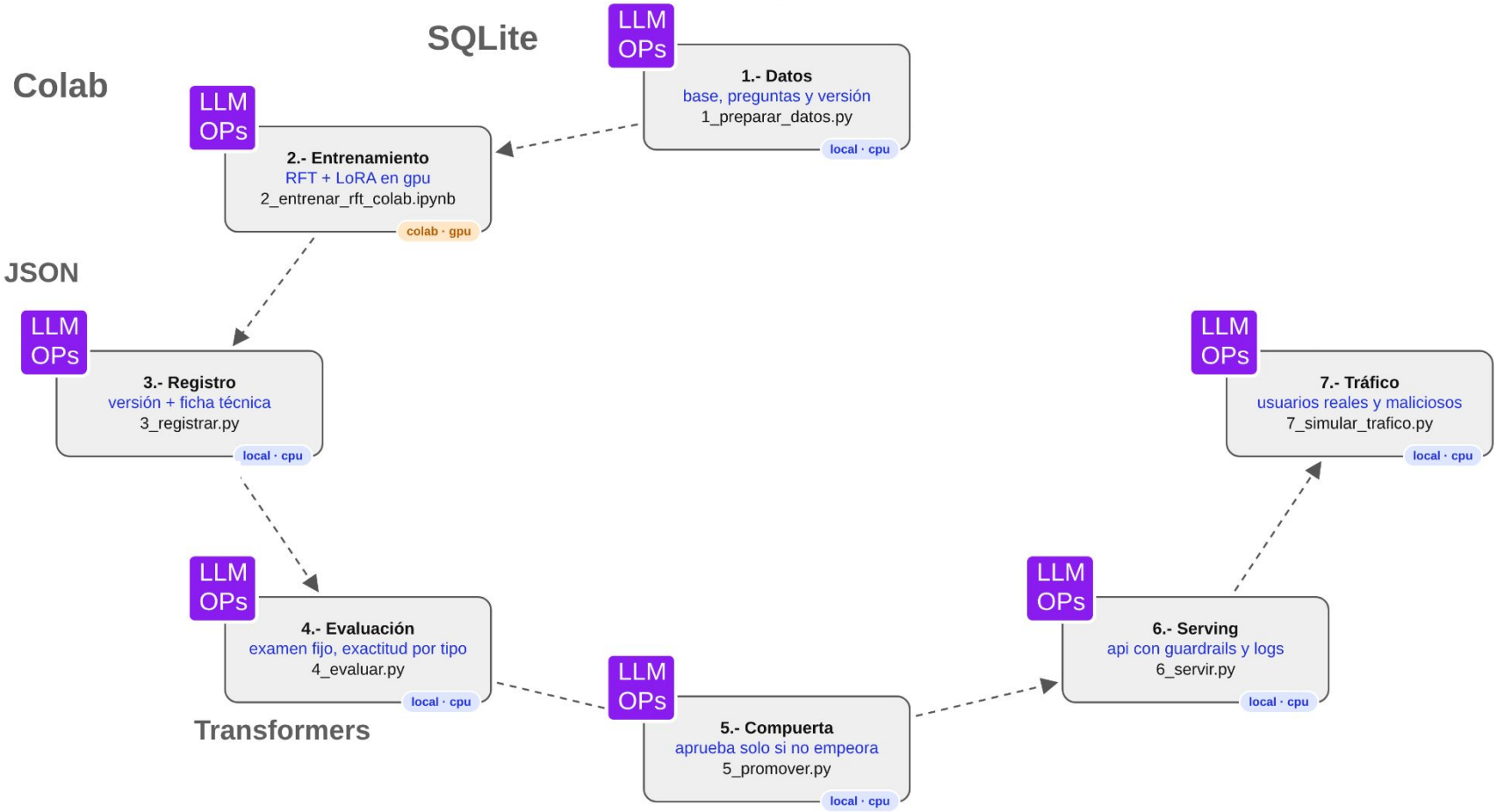
- exactitud ≥ 50 % OK
- no peor que v0 OK
- **v1 aprobada**

Flujo



la API carga v1. Cada consulta pasa por el guardrail (solo SELECT y conexión de solo lectura) y queda en el log

Flujo


SQLite
Colab
LLM OPs
1.- Datos
base, preguntas y versión
1_preparar_datos.py
local · cpu
LLM OPs
2.- Entrenamiento
RFT + LoRA en gpu
2_entrenar_rft_colab.ipynb
colab · gpu
JSON
LLM OPs
3.- Registro
versión + ficha técnica
3_registrar.py
local · cpu
LLM OPs
7.- Tráfico
usuarios reales y maliciosos
7_simular_trafico.py
local · cpu
LLM OPs
4.- Evaluación
examen fijo, exactitud por tipo
4_evaluar.py
local · cpu
Transformers
LLM OPs
5.- Compuerta
aprueba solo si no empeora
5_promover.py
local · cpu
LLM OPs
6.- Serving
api con guardrails y logs
6_servir.py
local · cpu

Flujo

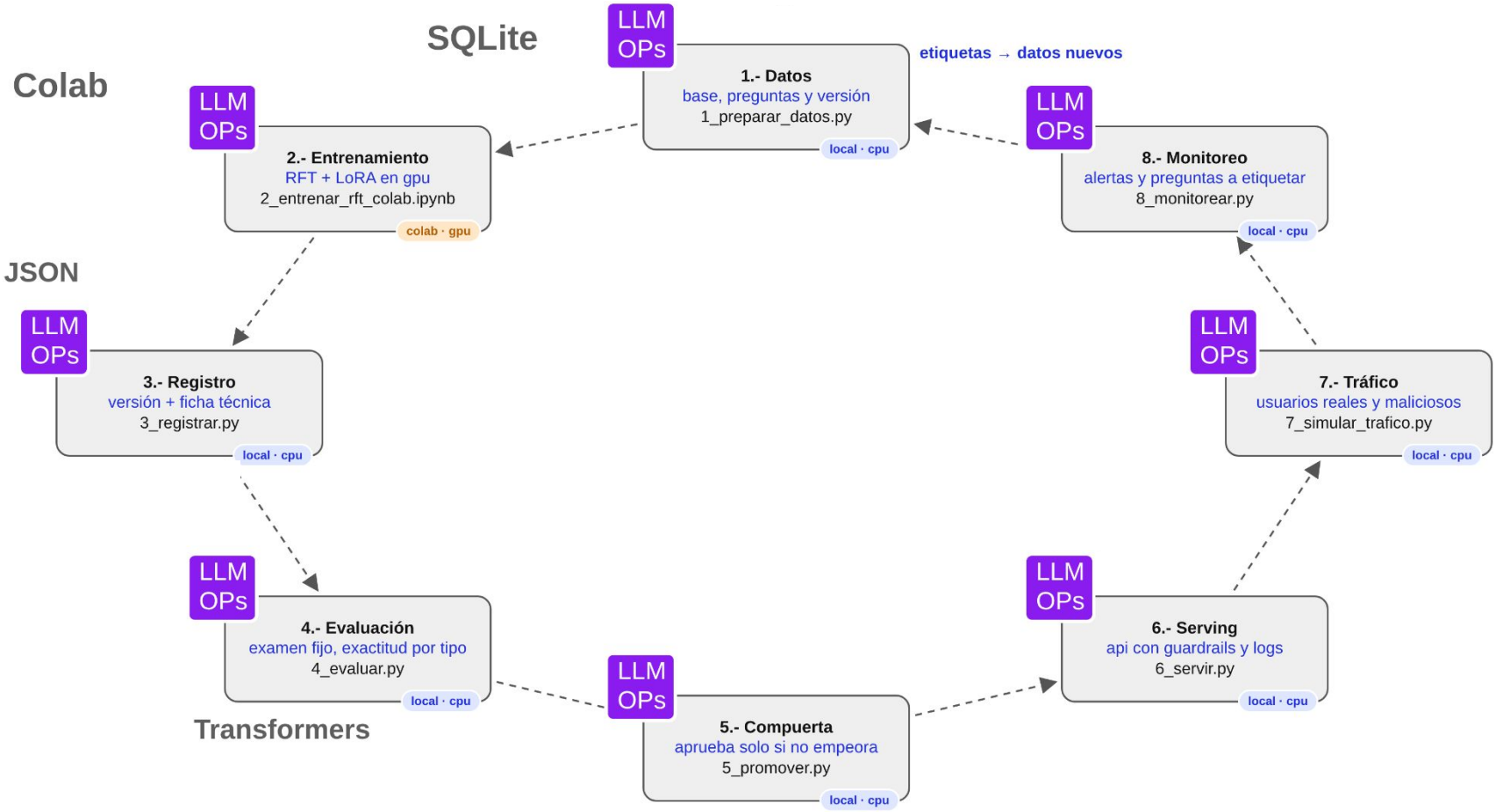
A: “¿Cuántos estudiantes aprobaron Estadística?”, que está en el examen.

B: “ciudades ordenadas por cantidad de estudiantes”, que llega desde producción.

Llega B, una pregunta redactada de una forma que el modelo nunca vio:

```
pregunta: "ciudades ordenadas por cantidad de estudiantes"
sql:      SELECT ciudad FROM estudiantes ORDER BY COUNT(*) DESC   # falta GROUP BY
estado:   error
```

Flujo


SQLite
Colab
JSON
Transformers
LLM OPs
1.- Datos
base, preguntas y versión
1_preparar_datos.py
local · cpu
etiquetas → datos nuevos
2.- Entrenamiento
RFT + LoRA en gpu
2_entrenar_rft_colab.ipynb
colab · gpu
3.- Registro
versión + ficha técnica
3_registrar.py
4.- Evaluación
examen fijo, exactitud por tipo
4_evaluar.py
5.- Compuerta
aprueba solo si no empeora
5_promover.py
6.- Serving
api con guardrails y logs
6_servir.py
7.- Tráfico
usuarios reales y maliciosos
7_simular_trafico.py
8.- Monitoreo
alertas y preguntas a etiquetar
8_monitorear.py

Flujo

A: “¿Cuántos estudiantes aprobaron Estadística?”, que está en el examen.

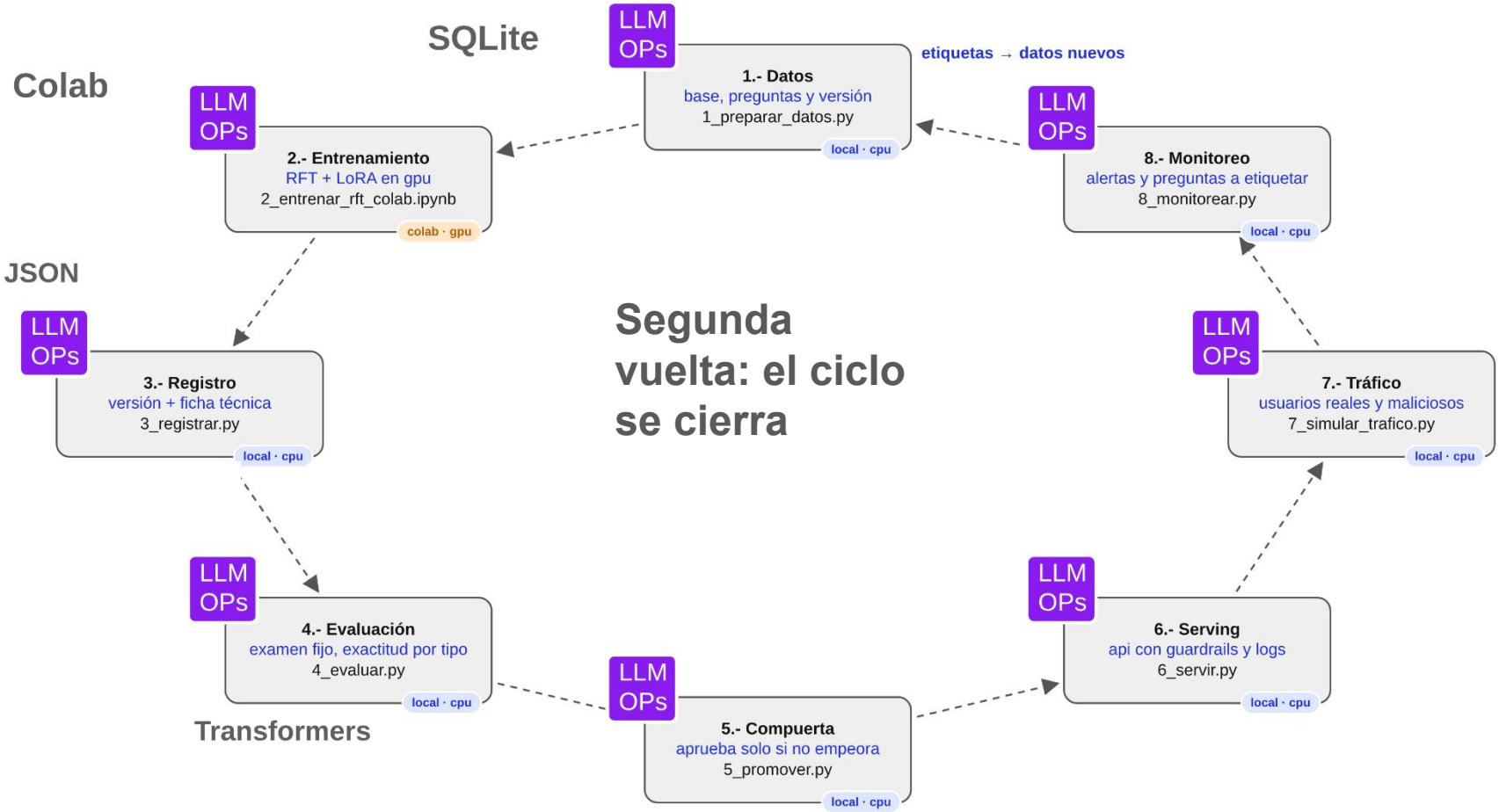
B: “ciudades ordenadas por cantidad de estudiantes”, que llega desde producción.

v1: 18 consultas , ok 89 % , error 1 , bloqueado 1
1 pregunta nueva para etiquetar en data/etiquetados.json

B queda en la lista, una persona escribe la SQL correcta:

{"pregunta": "ciudades ordenadas por cantidad de estudiantes",
"sql": "SELECT ciudad, COUNT(*) FROM estudiantes GROUP BY ciudad ORDER BY COUNT(*) DESC"}

Flujo


SQLite
Colab
JSON
Transformers
LLM OPs
1.- Datos
base, preguntas y versión
1_preparar_datos.py
local · cpu
etiquetas → datos nuevos
2.- Entrenamiento
RFT + LoRA en gpu
2_entrenar_rft_colab.ipynb
colab · gpu
3.- Registro
versión + ficha técnica
3_registrar.py
4.- Evaluación
examen fijo, exactitud por tipo
4_evaluar.py
5.- Compuerta
aprueba solo si no empeora
5_promover.py
6.- Serving
api con guardrails y logs
6_servir.py
7.- Tráfico
usuarios reales y maliciosos
7_simular_trafico.py
8.- Monitoreo
alertas y preguntas a etiquetar
8_monitorear.py
Segunda vuelta: el ciclo se cierra

Flujo

A: “¿Cuántos estudiantes aprobaron Estadística?”, que está en el examen.

B: “ciudades ordenadas por cantidad de estudiantes”, que llega desde producción.

1: B entra al entrenamiento, ahora hay 61 preguntas y la huella cambia a 7db03c52: es otra versión de los datos

2: Se entrenan tres variantes:

- v2 (RFT puro): en sus 8 intentos, el modelo nunca escribe un GROUP BY correcto, asi que B no aporta nada, en RFT la etiqueta sirve para juzgar no para enseñar
- v3 (RFT + respaldo): si ningún intento acierta, se entrena con la SQL etiquetada, ahora sí aprende el patrón

3 (registro, evaluación y compuerta):

Versión	A (aprobaron Estadística)	Exactitud	Compuerta
v2	NO olvidó nota >= 51	89 %	rechazada (cae “agregar”)
v3	OK	94 %	aprobada

Flujo

A: “¿Cuántos estudiantes aprobaron Estadística?”, que está en el examen.

B: “ciudades ordenadas por cantidad de estudiantes”, que llega desde producción.

4 (producción con v3)

```
pregunta: "ciudades ordenadas por cantidad de estudiantes"
sql:      SELECT ciudad, COUNT(*) FROM estudiantes GROUP BY ciudad
ORDER BY COUNT(*) DESC
estado:   ok
```

El monitoreo marca 0 errores, B completó el ciclo: falló en producción, se etiquetó, se reentrenó, pasó la compuerta y quedó corregida

El ciclo solo corrige lo que el monitoreo ve, las fallas necesitan revisión humana

Resumen

- **v0: modelo original (3_registrar.py)**
  - Qwen sin entrenar
  - acierta 28 % del examen
  - entra a producción solo como baseline
- **v1: RFT con los datos originales (notebook)**
  - original + adaptador entrenado con 59 preguntas
  - aprende solo de sus propios intentos correctos
  - acierta 94 %: **aprobada**
  - en producción falla “ciudades ordenadas” (le falta GROUP BY)
- **v2: RFT con datos + etiquetas (notebook)**
  - original + adaptador nuevo, con 61 preguntas (se suman las 2 que fallaron en producción)
  - no aprende las preguntas nuevas porque nunca acierta ninguno de sus 8 intentos
  - baja a 89 %: **rechazada** por la compuerta
- **v3: RFT + respaldo con etiqueta (notebook)**
  - mismos datos que v2
  - si falla los 8 intentos, entrena con la SQL correcta
  - acierta 94 %: **aprobada, en producción**

versión	contra	decisión
v1	v0	aprobada
v2	v1	rechazada
v3	v1	aprobada



Ejercicio

text-to-sql

LINK